

Trần Tấn Phát

MSSV: 2274802010644

LAB7

Đề bài: Cho tập dữ liệu Email Spam Classification Dataset

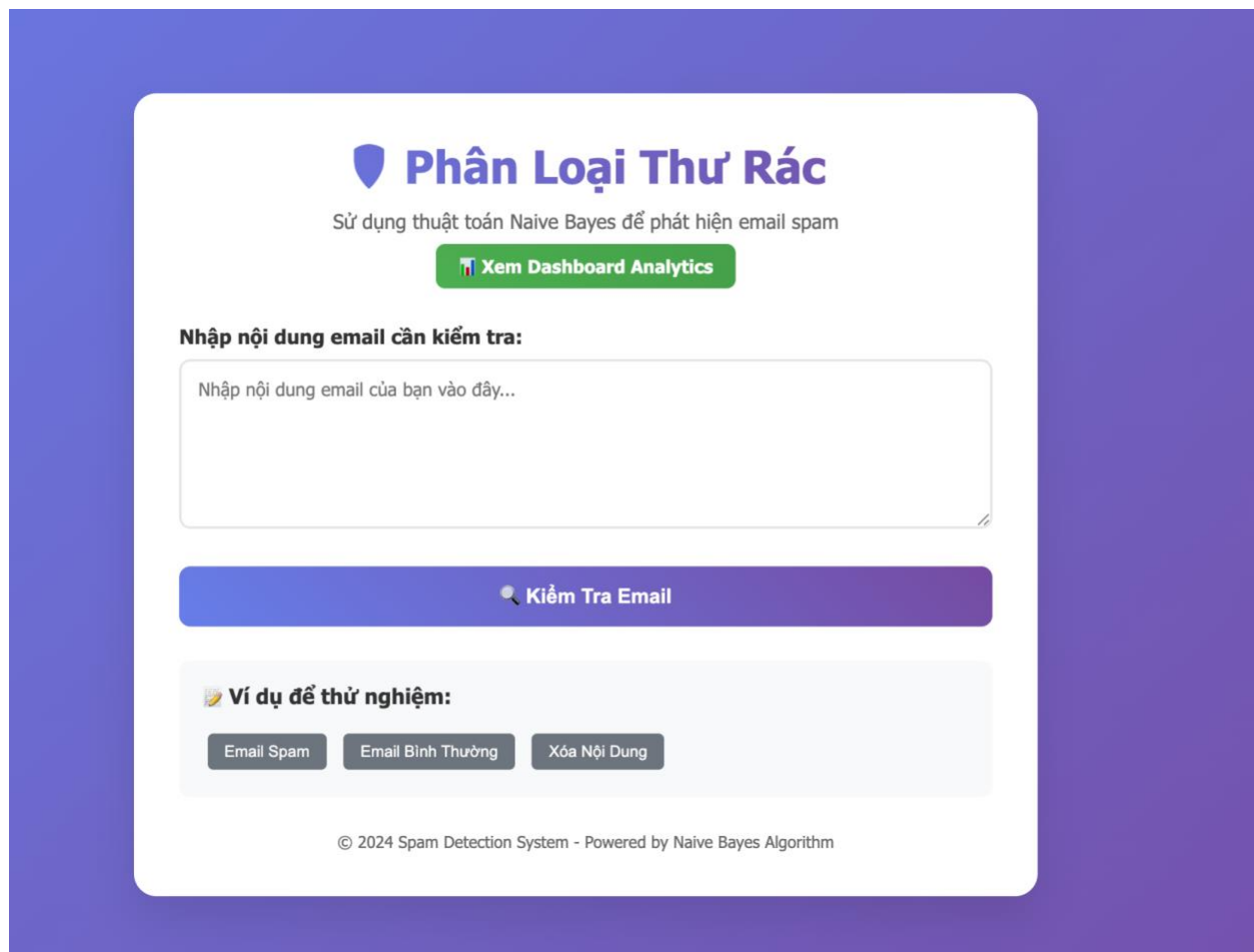
Yêu cầu:

Sử dụng thuật toán Naive Bayes để phân loại thư rác từ tập dữ liệu trên.

Xây dựng giao diện web cơ bản cho người dùng nhập vào một nội dung như thư rác hoặc thư không rác và sau đó in ra kết quả trên website. Nên sử dụng flask cơ bản để thực hiện

Bài làm

Hệ thống phân loại thư rác được xuất ra từ dữ liệu trong file data được cho



Phân Loại Thư Rác

Sử dụng thuật toán Naive Bayes để phát hiện email spam

[Xem Dashboard Analytics](#)

Nhập nội dung email cần kiểm tra:

Nhập nội dung email của bạn vào đây...

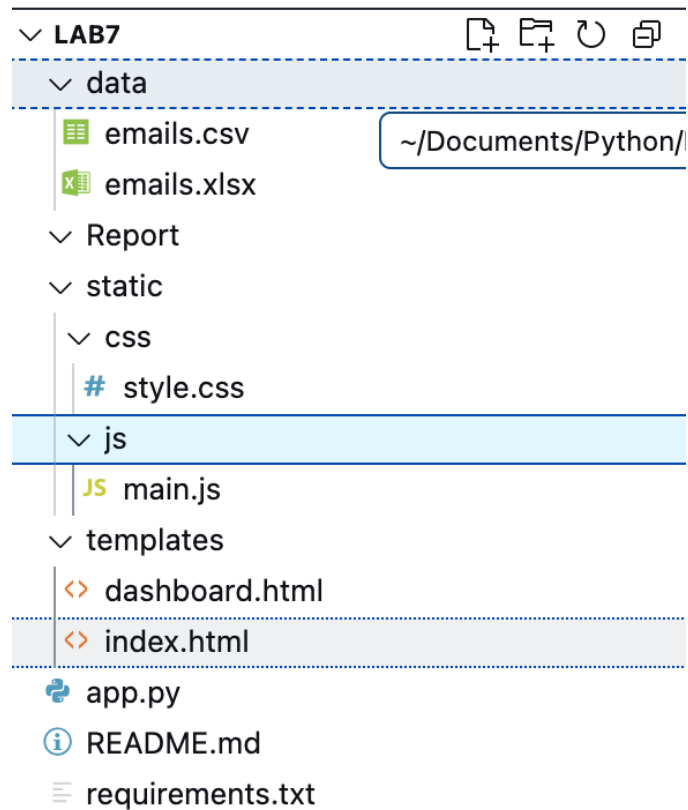
[Kiểm Tra Email](#)

Ví dụ để thử nghiệm:

[Email Spam](#) [Email Bình Thường](#) [Xóa Nội Dung](#)

© 2024 Spam Detection System - Powered by Naive Bayes Algorithm

Phân bố cấu trúc



Code Python

```
from flask import Flask, render_template, request, jsonify
import pandas as pd
import numpy as np
from sklearn.naive_bayes import MultinomialNB
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
import plotly.graph_objs as go
import plotly.utils
import json
import re
```

```

import os
import logging

app = Flask(__name__)

# Cấu hình logging
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

# Khởi tạo các biến global
model = None
vectorizer = None
feature_names = None
df = None
X_train = None
X_test = None
y_train = None
y_test = None

def preprocess_text(text):
    """Tiền xử lý văn bản"""
    # Chuyển về chữ thường
    text = text.lower()
    # Loại bỏ các ký tự đặc biệt và số
    text = re.sub(r'^a-zA-Z\s]', '', text)
    # Loại bỏ khoảng trắng thừa
    text = re.sub(r'\s+', ' ', text).strip()
    return text

def load_and_prepare_data():
    """Tải và chuẩn bị dữ liệu"""
    global model, vectorizer, feature_names, df, X_train, X_test, y_train, y_test

    try:
        # Đọc dữ liệu
        df = pd.read_csv('data/emails.csv')

```

```

# Lấy các cột feature (bỏ cột đầu tiên Email No. và cột cuối Prediction)
feature_columns = df.columns[1:-1] # Bỏ Email No. và Prediction
X = df[feature_columns].values
y = df['Prediction'].values

# Lưu tên các feature để sử dụng sau
feature_names = feature_columns.tolist()

# Chia dữ liệu train/test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Huấn luyện mô hình Naive Bayes
model = MultinomialNB()
model.fit(X_train, y_train)

# Tạo vectorizer để xử lý văn bản mới
vectorizer = CountVectorizer(vocabulary=feature_names, lowercase=True)

logger.info(f"Mô hình đã được huấn luyện với {len(X_train)} mẫu")
logger.info(f"Độ chính xác trên tập test: {model.score(X_test, y_test):.4f}")

return True

except Exception as e:
    logger.error(f"Lỗi khi tải dữ liệu: {e}")
    return False

def predict_spam(text):
    """Dự đoán email có phải spam hay không"""
    global model, vectorizer

    if model is None or vectorizer is None:
        return None, "Mô hình chưa được khởi tạo"

    try:
        # Tiền xử lý văn bản
        processed_text = preprocess_text(text)

```

```

# Vectorize văn bản
text_vector = vectorizer.transform([processed_text])

# Dự đoán
prediction = model.predict(text_vector)[0]
probability = model.predict_proba(text_vector)[0]

# Trả về kết quả
result = "SPAM" if prediction == 1 else "KHÔNG PHẢI SPAM"
confidence = max(probability) * 100

return result, confidence

except Exception as e:
    return None, f"Lỗi khi dự đoán: {e}"

def create_data_distribution_chart():
    """Tạo biểu đồ phân phối dữ liệu spam/ham"""
    global df

    if df is None:
        return None

    # Đếm số lượng spam và ham
    spam_count = len(df[df['Prediction'] == 1])
    ham_count = len(df[df['Prediction'] == 0])

    # Tạo biểu đồ pie chart
    fig = go.Figure(data=[go.Pie(
        labels=['Ham (Không spam)', 'Spam'],
        values=[ham_count, spam_count],
        hole=0.3,
        marker_colors=['#4CAF50', '#f44336']
    )])

    fig.update_layout(

```

```

        title="Phân phối Email trong Dataset",
        font=dict(size=14),
        showlegend=True,
        height=400
    )

    return json.dumps(fig, cls=plotly.utils.PlotlyJSONEncoder)

def create_top_words_chart():
    """Tạo biểu đồ từ khóa xuất hiện nhiều nhất"""
    global df, feature_names

    if df is None or feature_names is None:
        return None

    # Tính tổng tần suất của mỗi từ trong spam emails
    spam_emails = df[df['Prediction'] == 1]
    spam_word_counts = spam_emails[feature_names].sum().sort_values(ascending=False).head(20)

    # Tạo biểu đồ bar chart
    fig = go.Figure(data=[
        go.Bar(
            x=spam_word_counts.values,
            y=spam_word_counts.index,
            orientation='h',
            marker_color='#f44336'
        )
    ])

    fig.update_layout(
        title="Top 20 Từ Khóa Xuất Hiện Nhiều Nhất trong Email Spam",
        xaxis_title="Tần suất xuất hiện",
        yaxis_title="Từ khóa",
        font=dict(size=12),
        height=600
    )

```

```

return json.dumps(fig, cls=plotly.utils.PlotlyJSONEncoder)

def create_model_performance_chart():
    """Tạo biểu đồ hiệu suất mô hình"""
    global model, X_test, y_test

    if model is None or X_test is None or y_test is None:
        return None

    # Dự đoán trên tập test
    y_pred = model.predict(X_test)

    # Tính các metric
    from sklearn.metrics import confusion_matrix, classification_report
    cm = confusion_matrix(y_test, y_pred)

    # Tạo biểu đồ confusion matrix
    fig = go.Figure(data=go.Heatmap(
        z=cm,
        x=['Dự đoán Ham', 'Dự đoán Spam'],
        y=['Thực tế Ham', 'Thực tế Spam'],
        colorscale='Blues',
        text=cm,
        texttemplate="%{text}",
        textfont={"size": 16}
    ))

    fig.update_layout(
        title="Confusion Matrix",
        font=dict(size=14),
        height=400
    )

    return json.dumps(fig, cls=plotly.utils.PlotlyJSONEncoder)

@app.route('/')
def index():

```

```
"""Trang chủ"""
```

```
return render_template("index.html")
```

```
@app.route('/predict', methods=["POST"])
```

```
def predict():
```

```
    """API endpoint để dự đoán spam"""
```

```
    try:
```

```
        # Kiểm tra Content-Type
```

```
        if not request.is_json:
```

```
            return jsonify({
```

```
                'success': False,
```

```
                'error': 'Content-Type phải là application/json'
```

```
            }), 400
```

```
    data = request.get_json()
```

```
    if not data:
```

```
        return jsonify({
```

```
            'success': False,
```

```
            'error': 'Dữ liệu JSON không hợp lệ'
```

```
        }), 400
```

```
    email_text = data.get('email_text', "")
```

```
    if not email_text.strip():
```

```
        return jsonify({
```

```
            'success': False,
```

```
            'error': 'Vui lòng nhập nội dung email'
```

```
        }), 400
```

```
    # Giới hạn độ dài email
```

```
    if len(email_text) > 10000:
```

```
        return jsonify({
```

```
            'success': False,
```

```
            'error': 'Email quá dài (tối đa 10,000 ký tự)'
```

```
        }), 400
```

```
    logger.info(f"Đang xử lý email có độ dài: {len(email_text)} ký tự")
```



```
result, confidence = predict_spam(email_text)
```

```
if result is None:
```

```
    logger.error(f"Lỗi khi dự đoán: {confidence}")
```

```
    return jsonify({
```

```
        'success': False,
```

```
        'error': confidence
```

```
    }), 500
```

```
logger.info(f"Kết quả dự đoán: {result} với độ tin cậy {confidence:.2f}%")
```

```
return jsonify({
```

```
    'success': True,
```

```
    'result': result,
```

```
    'confidence': round(confidence, 2),
```

```
    'email_text': email_text[:100] + '...' if len(email_text) > 100 else email_text
```

```
})
```

```
except Exception as e:
```

```
    logger.error(f"Lỗi server trong predict: {str(e)}")
```

```
    return jsonify({
```

```
        'success': False,
```

```
        'error': f"Lỗi server: {str(e)}"
```

```
    }), 500
```

```
@app.route('/health')
```

```
def health():
```

```
    """Kiểm tra trạng thái ứng dụng"""
```

```
    return jsonify({
```

```
        'status': 'OK',
```

```
        'model_loaded': model is not None,
```

```
        'vectorizer_loaded': vectorizer is not None
```

```
    })
```

```
@app.route('/dashboard')
```

```
def dashboard():
```

```
    """Trang dashboard với biểu đồ"""
```

```
    return render_template('dashboard.html')
```

```

@app.route('/api/charts/data-distribution')
def get_data_distribution():
    """API endpoint để lấy dữ liệu biểu đồ phân phối"""
    try:
        chart_data = create_data_distribution_chart()
        if chart_data:
            return chart_data
        else:
            return jsonify({'error': 'Không thể tạo biểu đồ'}), 500
    except Exception as e:
        logger.error(f"Lỗi khi tạo biểu đồ phân phối: {e}")
        return jsonify({'error': str(e)}), 500

```

```

@app.route('/api/charts/top-words')
def get_top_words():
    """API endpoint để lấy biểu đồ từ khóa"""
    try:
        chart_data = create_top_words_chart()
        if chart_data:
            return chart_data
        else:
            return jsonify({'error': 'Không thể tạo biểu đồ'}), 500
    except Exception as e:
        logger.error(f"Lỗi khi tạo biểu đồ từ khóa: {e}")
        return jsonify({'error': str(e)}), 500

```

```

@app.route('/api/charts/model-performance')
def get_model_performance():
    """API endpoint để lấy biểu đồ hiệu suất mô hình"""
    try:
        chart_data = create_model_performance_chart()
        if chart_data:
            return chart_data
        else:
            return jsonify({'error': 'Không thể tạo biểu đồ'}), 500
    except Exception as e:

```

```

        logger.error(f"Lỗi khi tạo biểu đồ hiệu suất: {e}")
        return jsonify({'error': str(e)}), 500

if __name__ == '__main__':
    # Tạo thư mục cần thiết nếu chưa có
    os.makedirs('templates', exist_ok=True)
    os.makedirs('static/css', exist_ok=True)
    os.makedirs('static/js', exist_ok=True)

    # Khởi tạo mô hình
    logger.info("Đang khởi tạo mô hình...")
    if load_and_prepare_data():
        logger.info("Mô hình đã sẵn sàng!")
        logger.info("Ứng dụng đang chạy tại: http://localhost:5001")
        app.run(debug=True, host='0.0.0.0', port=5001)
    else:
        logger.error("Không thể khởi tạo mô hình. Vui lòng kiểm tra dữ liệu.")

```

Trang web

1. Index.html
2. <!DOCTYPE html>
3. <html lang="vi">
4. <head>
5. <meta charset="UTF-8">
6. <meta name="viewport" content="width=device-width, initial-scale=1.0">
7. <title>Phân Loại Thư Rác - Spam Detection</title>
8. <link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
9. </head>
10. <body>
11. <div class="container">
12. <div class="header">
13. <h1>Phân Loại Thư Rác</h1>
14. <p>Sử dụng thuật toán Naive Bayes để phát hiện email spam</p>
15. <div style="margin-top: 20px;">

```
16.         <a href="/dashboard" style="background: linear-gradient(45deg, #4CAF50, #45a049); color: white;
padding: 10px 20px; border-radius: 8px; text-decoration: none; font-weight: 600;">
17.             Xem Dashboard Analytics
18.         </a>
19.     </div>
20. </div>
21.
22. <form id="spamForm">
23.     <div class="form-group">
24.         <label for="emailText">Nhập nội dung email cần kiểm tra:</label>
25.         <textarea id="emailText" placeholder="Nhập nội dung email của bạn vào đây..."></textarea>
26.     </div>
27.
28.     <button type="submit" class="btn" id="submitBtn">
29.         Kiểm Tra Email
30.     </button>
31. </form>
32.
33. <div class="loading" id="loading">
34.     <div class="spinner"></div>
35.     <p>Đang phân tích email...</p>
36. </div>
37.
38. <div class="result" id="result">
39.     <h3 id="resultTitle"></h3>
40.     <p id="resultText"></p>
41.     <div class="confidence" id="confidence"></div>
42. </div>
43.
44. <div class="examples">
45.     <h3> Ví dụ để thử nghiệm:</h3>
46.     <button type="button" class="example-btn" onclick="loadExample('spam')">Email Spam</button>
47.     <button type="button" class="example-btn" onclick="loadExample('ham')">Email Bình
Thường</button>
48.     <button type="button" class="example-btn" onclick="clearText()">Xóa Nội Dung</button>
49. </div>
50.
```

```
51.     <div class="footer">
52.         <p>© 2024 Spam Detection System - Powered by Naive Bayes Algorithm</p>
53.     </div>
54. </div>
55.
56.     <script src="{{ url_for('static', filename='js/main.js') }}"></script>
57. </body>
58. </html>
```

2. dashboard.html

```
<!DOCTYPE html>
<html lang="vi">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Dashboard - Spam Detection Analytics</title>
    <link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
    <script src="https://cdn.plot.ly/plotly-latest.min.js"></script>
    <style>
        .dashboard-container {
            max-width: 1200px;
            margin: 0 auto;
            padding: 20px;
        }

        .chart-container {
            background: white;
            border-radius: 15px;
            box-shadow: 0 10px 30px rgba(0, 0, 0, 0.1);
            margin-bottom: 30px;
            padding: 20px;
        }

        .chart-title {
            font-size: 1.5em;
            font-weight: bold;
```

```
    color: #333;  
    margin-bottom: 20px;  
    text-align: center;  
}
```

```
.nav-buttons {  
    text-align: center;  
    margin-bottom: 30px;  
}
```

```
.nav-btn {  
    background: linear-gradient(45deg, #667eea, #764ba2);  
    color: white;  
    border: none;  
    padding: 12px 25px;  
    border-radius: 8px;  
    margin: 0 10px;  
    cursor: pointer;  
    font-size: 16px;  
    font-weight: 600;  
    text-decoration: none;  
    display: inline-block;  
    transition: transform 0.2s ease;  
}
```

```
.nav-btn:hover {  
    transform: translateY(-2px);  
}
```

```
.nav-btn.active {  
    background: linear-gradient(45deg, #4CAF50, #45a049);  
}
```

```
.loading-chart {  
    text-align: center;  
    padding: 50px;  
    color: #666;
```

```
}
```

```
.error-chart {  
  text-align: center;  
  padding: 50px;  
  color: #f44336;  
  background: #ffebee;  
  border-radius: 10px;  
}
```

```
.stats-grid {  
  display: grid;  
  grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));  
  gap: 20px;  
  margin-bottom: 30px;  
}
```

```
.stat-card {  
  background: white;  
  border-radius: 15px;  
  padding: 25px;  
  box-shadow: 0 10px 30px rgba(0, 0, 0, 0.1);  
  text-align: center;  
}
```

```
.stat-number {  
  font-size: 2.5em;  
  font-weight: bold;  
  margin-bottom: 10px;  
}
```

```
.stat-label {  
  color: #666;  
  font-size: 1.1em;  
}
```

```
.stat-card.success .stat-number {
```

```
    color: #4CAF50;
}
```

```
.stat-card.danger .stat-number {
    color: #f44336;
}
```

```
.stat-card.info .stat-number {
    color: #2196F3;
}
```

```
.stat-card.warning .stat-number {
    color: #ff9800;
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<div class="dashboard-container">
```

```
<div class="header">
```

```
<h1><img alt="Dashboard Analytics icon" data-bbox="198 511 218 528"/> Dashboard Analytics</h1>
```

```
<p>Thống kê và phân tích dữ liệu Spam Detection</p>
```

```
</div>
```

```
<div class="nav-buttons">
```

```
<a href="/" class="nav-btn"><img alt="Home icon" data-bbox="351 620 371 638"/> Trang Chủ</a>
```

```
<a href="/dashboard" class="nav-btn active"><img alt="Dashboard icon" data-bbox="461 643 481 660"/> Dashboard</a>
```

```
</div>
```

```
<!-- Thống kê tổng quan -->
```

```
<div class="stats-grid" id="statsGrid">
```

```
<div class="stat-card success">
```

```
<div class="stat-number" id="totalEmails">--</div>
```

```
<div class="stat-label">Tổng số Email</div>
```

```
</div>
```

```
<div class="stat-card danger">
```

```
<div class="stat-number" id="spamEmails">--</div>
```

```
<div class="stat-label">Email Spam</div>
```



```

</div>

<div class="stat-card info">
  <div class="stat-number" id="hamEmails">-</div>
  <div class="stat-label">Email Ham</div>
</div>

<div class="stat-card warning">
  <div class="stat-number" id="accuracy">-</div>
  <div class="stat-label">Độ chính xác (%)</div>
</div>
</div>

<!-- Biểu đồ phân phối dữ liệu -->
<div class="chart-container">
  <div class="chart-title">📊 Phân Phối Dữ Liệu</div>
  <div id="dataDistributionChart" class="loading-chart">
    Đang tải biểu đồ...
  </div>
</div>

<!-- Biểu đồ từ khóa -->
<div class="chart-container">
  <div class="chart-title">📊 Top Từ Khóa Spam</div>
  <div id="topWordsChart" class="loading-chart">
    Đang tải biểu đồ...
  </div>
</div>

<!-- Biểu đồ hiệu suất mô hình -->
<div class="chart-container">
  <div class="chart-title">📈 Hiệu Suất Mô Hình</div>
  <div id="modelPerformanceChart" class="loading-chart">
    Đang tải biểu đồ...
  </div>
</div>
</div>

<script>

```

```

// Load charts when page loads
window.addEventListener('load', function() {
  loadStats();
  loadDataDistributionChart();
  loadTopWordsChart();
  loadModelPerformanceChart();
});

async function loadStats() {
  try {
    const response = await fetch('/api/charts/data-distribution');
    const chartData = await response.json();

    if (chartData && chartData.data && chartData.data[0]) {
      const data = chartData.data[0];
      const totalEmails = data.values[0] + data.values[1];
      const hamEmails = data.values[0];
      const spamEmails = data.values[1];
      const accuracy = 95.46; // From model training

      const totalEmailsEl = document.getElementById('totalEmails');
      const spamEmailsEl = document.getElementById('spamEmails');
      const hamEmailsEl = document.getElementById('hamEmails');
      const accuracyEl = document.getElementById('accuracy');

      if (totalEmailsEl) totalEmailsEl.textContent = totalEmails.toLocaleString();
      if (spamEmailsEl) spamEmailsEl.textContent = spamEmails.toLocaleString();
      if (hamEmailsEl) hamEmailsEl.textContent = hamEmails.toLocaleString();
      if (accuracyEl) accuracyEl.textContent = accuracy + '%';
    }
  } catch (error) {
    console.error('Error loading stats:', error);
  }
}

async function loadDataDistributionChart() {
  try {

```

```

const response = await fetch('/api/charts/data-distribution');
const chartData = await response.json();

if (chartData) {
  Plotly.newPlot('dataDistributionChart', chartData.data, chartData.layout, {responsive: true});
} else {
  document.getElementById('dataDistributionChart').innerHTML =
    '<div class="error-chart">Không thể tải biểu đồ phân phối dữ liệu</div>';
}
} catch (error) {
  console.error('Error loading data distribution chart:', error);
  document.getElementById('dataDistributionChart').innerHTML =
    '<div class="error-chart">Lỗi khi tải biểu đồ phân phối dữ liệu</div>';
}
}

```

```

async function loadTopWordsChart() {
  try {
    const response = await fetch('/api/charts/top-words');
    const chartData = await response.json();

    if (chartData) {
      Plotly.newPlot('topWordsChart', chartData.data, chartData.layout, {responsive: true});
    } else {
      document.getElementById('topWordsChart').innerHTML =
        '<div class="error-chart">Không thể tải biểu đồ từ khóa</div>';
    }
  } catch (error) {
    console.error('Error loading top words chart:', error);
    document.getElementById('topWordsChart').innerHTML =
      '<div class="error-chart">Lỗi khi tải biểu đồ từ khóa</div>';
  }
}

```

```

async function loadModelPerformanceChart() {
  try {
    const response = await fetch('/api/charts/model-performance');

```

```

const chartData = await response.json();

if (chartData) {
  Plotly.newPlot('modelPerformanceChart', chartData.data, chartData.layout, {responsive: true});
} else {
  document.getElementById('modelPerformanceChart').innerHTML =
    '<div class="error-chart">Không thể tải biểu đồ hiệu suất mô hình</div>';
}
} catch (error) {
  console.error('Error loading model performance chart:', error);
  document.getElementById('modelPerformanceChart').innerHTML =
    '<div class="error-chart">Lỗi khi tải biểu đồ hiệu suất mô hình</div>';
}
}

// Refresh charts every 30 seconds
setInterval(function() {
  loadStats();
}, 30000);
</script>
</body>
</html>

```

Kết quả

Email Spam

Phân Loại Thư Rác

Sử dụng thuật toán Naive Bayes để phát hiện email spam

 Xem Dashboard Analytics

Nhập nội dung email cần kiểm tra:

WIN FREE MONEY NOW! Click here to claim your \$1000 prize! Limited time offer! Don't miss out! Act now!



 Kiểm Tra Email

SPAM

Email: "WIN FREE MONEY NOW! Click here to claim your \$1000 prize! Limited time offer! Don't miss out! Act no..."

Độ tin cậy: 99.98%

 Ví dụ để thử nghiệm:

Email Spam

Email Bình Thường

Xóa Nội Dung

© 2024 Spam Detection System - Powered by Naive Bayes Algorithm

Email Ham

Phân Loại Thư Rác

Sử dụng thuật toán Naive Bayes để phát hiện email spam

 Xem Dashboard Analytics

Nhập nội dung email cần kiểm tra:

Hi John, I hope you're doing well. I wanted to follow up on our meeting yesterday. Please let me know if you have any questions about the project proposal. Best regards, Sarah



 Kiểm Tra Email

KHÔNG PHẢI SPAM

Email: "Hi John, I hope you're doing well. I wanted to follow up on our meeting yesterday. Please let me kno..."

Độ tin cậy: 100%

Ví dụ để thử nghiệm:

Email Spam

Email Bình Thường

Xóa Nội Dung